

# Dokumentacja bezpieczeństwa:

## 1. Przegląd ogólny

Aplikacja została sprawdzona w serwisie Security Headers, gdzie początkowo otrzymała ocenę D. Wynik ten wskazywał na brak kilku podstawowych nagłówków bezpieczeństwa.

W ramach poprawy bezpieczeństwa wdrożono rozwiązania obejmujące nagłówki HTTP, bezpieczne przechowywanie sekretów w środowisku chmurowym oraz mechanizm ograniczania liczby zapytań do backendu.

### 1.1. Cel wprowadzonych zmian

Celem zmian było zwiększenie bezpieczeństwa aplikacji oraz ograniczenie ryzyka typowych problemów występujących w aplikacjach webowych.

W szczególności skupiono się na:

- poprawie konfiguracji nagłówków bezpieczeństwa,
- ochronie danych konfiguracyjnych i sekretów,
- ograniczeniu nadmiernej liczby zapytań do API,
- poprawie wyniku w narzędziu Security Headers.

### 1.2. Zakres zabezpieczeń

Zastosowane zabezpieczenia objęły:

- frontend hostowany w Firebase Hosting,
- backend aplikacji działający w Javie,
- konfigurację środowiska chmurowego,
- endpointy API narażone na częste zapytania.

## 2. Nagłówki bezpieczeństwa HTTP

Nagłówki bezpieczeństwa HTTP informują przeglądarkę, w jaki sposób ma obsługiwać stronę oraz jakie zachowania powinny zostać ograniczone lub zablokowane.

W projekcie uwzględniono kilka podstawowych nagłówków poprawiających bezpieczeństwo aplikacji.

### 2.1. X-Frame-Options

Nagłówek X-Frame-Options chroni aplikację przed osadzaniem jej w ramach iframe na obcych stronach.

Zmniejsza to ryzyko ataku typu clickjacking, w którym użytkownik mógłby kliknąć ukryty element aplikacji bez swojej wiedzy.

Zastosowane ustawienie:

- SAMEORIGIN

Oznacza ono, że strona może być osadzana tylko w obrębie tej samej domeny.

## 2.2. X-Content-Type-Options

Nagłówek X-Content-Type-Options zapobiega samodzielnemu zgadywaniu typu pliku przez przeglądarkę.

Zastosowane ustawienie:

- nosniff

Dzięki temu przeglądarka korzysta z typu pliku zadeklarowanego przez serwer, co ogranicza ryzyko błędnej interpretacji plików.

## 2.3. Referrer-Policy

Nagłówek Referrer-Policy kontroluje, jakie informacje o poprzednim adresie strony są przekazywane podczas przechodzenia na inne strony.

Zastosowane ustawienie:

- strict-origin-when-cross-origin

Oznacza to, że przy przechodzeniu między różnymi domenami przekazywana jest tylko podstawowa informacja o źródle, bez pełnego adresu URL.

## 2.4. Permissions-Policy

Nagłówek Permissions-Policy ogranicza dostęp aplikacji do wybranych funkcji przeglądarki.

W projekcie zablokowano nieużywane uprawnienia:

- geolocation=()
- microphone=()
- camera=()

Dzięki temu aplikacja nie uzyskuje dostępu do kamery, mikrofonu ani lokalizacji, jeśli nie są one potrzebne do jej działania.

## 2.5. Content-Security-Policy

Content-Security-Policy, czyli CSP, pozwala określić, z jakich źródeł aplikacja może ładować skrypty, style, obrazy i inne zasoby.

W projekcie zastosowano ostrożną konfigurację CSP, ponieważ zbyt restrykcyjne ustawienia mogłyby zakłócić działanie frontendu, szczególnie w przypadku zewnętrznych bibliotek, stylów lub zasobów.

CSP zwiększa bezpieczeństwo aplikacji, ograniczając możliwość ładowania nieautoryzowanych zasobów.

## 3. Zmienne środowiskowe i sekrety w chmurze

W projekcie dane wrażliwe nie są przechowywane bezpośrednio w kodzie źródłowym. Zamiast tego wykorzystywane są zmienne środowiskowe oraz sekrety dostępne w środowisku chmurowym.

### 3.1. Cel stosowania zmiennych środowiskowych

Zmienne środowiskowe pozwalają oddzielić konfigurację aplikacji od kodu źródłowego.

Dzięki temu dane poufne nie trafiają do repozytorium i nie są widoczne dla osób, które nie powinny mieć do nich dostępu.

### 3.2. Przykłady danych przechowywanych poza kodem

Poza kodem źródłowym mogą być przechowywane między innymi:

- adres bazy danych,
- dane dostępowe do usług zewnętrznych,
- tokeny autoryzacyjne,
- klucze API,
- ustawienia środowiska produkcyjnego.

### 3.3. Korzyści z takiego rozwiązania

Takie podejście zwiększa bezpieczeństwo projektu, ponieważ nawet w przypadku udostępnienia kodu źródłowego sekrety nie zostają ujawnione.

Dodatkowo pozwala to łatwo rozdzielić konfigurację lokalną, testową i produkcyjną bez potrzeby zmieniania kodu aplikacji.

## 4. Rate limiting

Rate limiting to mechanizm ograniczający liczbę zapytań, które użytkownik lub bot może wysłać do aplikacji w określonym czasie.

### 4.1. Cel rate limitingu

Celem rate limitingu jest ochrona backendu przed przeciążeniem oraz ograniczenie prostych prób nadużywania API.

Mechanizm ten pomaga zabezpieczyć aplikację przed masowym wysyłaniem zapytań, np. przez boty.

## 4.2. Miejsca zastosowania

Rate limiting może być stosowany szczególnie w miejscach, które są często wykorzystywane lub bardziej narażone na nadużycia.

Dotyczy to między innymi:

- logowania,
- rejestracji,
- wysyłania formularzy,
- dodawania postów,
- dodawania opinii,
- wybranych endpointów API.

## 4.3. Działanie mechanizmu

Jeżeli użytkownik lub bot przekroczy dozwoloną liczbę zapytań w krótkim czasie, backend może odrzucić kolejne żądania i zwrócić odpowiedni komunikat błędu.

Dzięki temu jedna osoba lub automatyczny skrypt nie może wysyłać nieograniczonej liczby zapytań do aplikacji.

## 4.4. Efekt zastosowania

Rate limiting poprawia stabilność aplikacji i zmniejsza ryzyko przeciążenia backendu.

Dodatkowo ogranicza możliwość automatycznego testowania formularzy, logowania lub innych endpointów API.

# 5. Podsumowanie

Wprowadzone rozwiązania stanowią podstawowy zestaw zabezpieczeń dla aplikacji webowej.

## 5.1. Najważniejsze korzyści

Zastosowane mechanizmy:

- poprawiają wynik w narzędziu Security Headers,
- ograniczają ryzyko clickjackingu,
- zmniejszają ryzyko błędnej interpretacji plików przez przeglądarkę,
- ograniczają ujawnianie pełnych adresów URL,
- blokują nieużywane uprawnienia przeglądarki,
- chronią sekrety i dane konfiguracyjne,
- ograniczają nadmierną liczbę zapytań do backendu.

## 5.2. Wnioski

Zastosowane zabezpieczenia nie zastępują pełnego audytu bezpieczeństwa, ale stanowią ważną podstawę ochrony aplikacji.

Dzięki nim projekt lepiej spełnia podstawowe wymagania bezpieczeństwa oraz pokazuje zastosowanie dobrych praktyk przy tworzeniu aplikacji webowej.